

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет программной инженерии и компьютерной техники Направление подготовки

09.03.04 Программная инженерия

Дисциплина «Бизнес-логика программных систем»

ОТЧЕТ

По лабораторной работе 3

Вариант 181

Преподаватель:

Кривоносов Егор Дмитриевич

Выполнил:

Алхимовици Арсений, Юсупова Алиса

Группа:

Р3310, 3313

Санкт-Петербург

2026 г.

Текст задания

Доработать приложение из лабораторной работы 2, реализовав в нём асинхронное выполнение задач с распределением бизнес-логики между несколькими вычислительными узлами и выполнением периодических операций с использованием планировщика задач, а также интеграцию с внешней информационной системой.

Требования к реализации асинхронной обработки:

Перед выполнением работы необходимо согласовать с преподавателем набор прецедентов, в реализации которых целесообразно использование асинхронного распределённого выполнения задач. Если таких прецедентов использования в имеющейся бизнес-процесса нет, нужно согласовать реализацию новых прецедентов, доработав таким образом модель бизнес-процесса из лабораторной работы 1. Асинхронное выполнение задач должно использовать модель доставки “очередь сообщений”. В качестве провайдера сервиса асинхронного обмена сообщениями необходимо использовать очередь сообщений на базе RabbitMQ. Для отправки сообщений необходимо использовать протокол MQTT. Библиотеку для реализации отправки сообщений можно взять любую на выбор студента. Для получения сообщений необходимо использовать JMS API. Требования к реализации распределённой обработки:

Обработка сообщений должна осуществляться на двух независимых друг от друга узлах сервера приложений. Если логика сценария распределённой обработки предполагает транзакционность выполняемых операций, они должны быть включены в состав распределённой транзакции. Требования к реализации запуска периодических задач по расписанию:

Согласовать с преподавателем прецедент или прецеденты, в рамках которых выглядит целесообразным использовать планировщик задач. Если такие прецеденты отсутствуют – согласовать с преподавателем новые и добавить их в модель автоматизируемого бизнес-процесса. Реализовать утверждённые прецеденты с использованием планировщика задач Spring (Scheduled). Требования к интеграции с внешней Корпоративной Информационной Системой (EIS):

Корпоративная Информационная Система, с которой производится интеграция, а также её функциональные возможности выбираются на усмотрение преподавателя и согласуются с ним. Взаимодействие с внешней Корпоративной Информационной Системой должно быть реализовано с помощью технологии JCA (Jakarta Connectors). Правила выполнения работы:

Все изменения, внесённые в реализуемый бизнес-процесс, должны быть учтены в описывающей его модели, REST API и наборе скриптов для тестирования публичных интерфейсов модуля. Доработанное приложение необходимо либо развернуть на сервере helios, либо продемонстрировать его работоспособность на собственной инфраструктуре обучающегося.

Исходный код

<https://github.com/senya-2011/mts-online-shop>

Модель потока управления для автоматизируемого бизнес-процесса

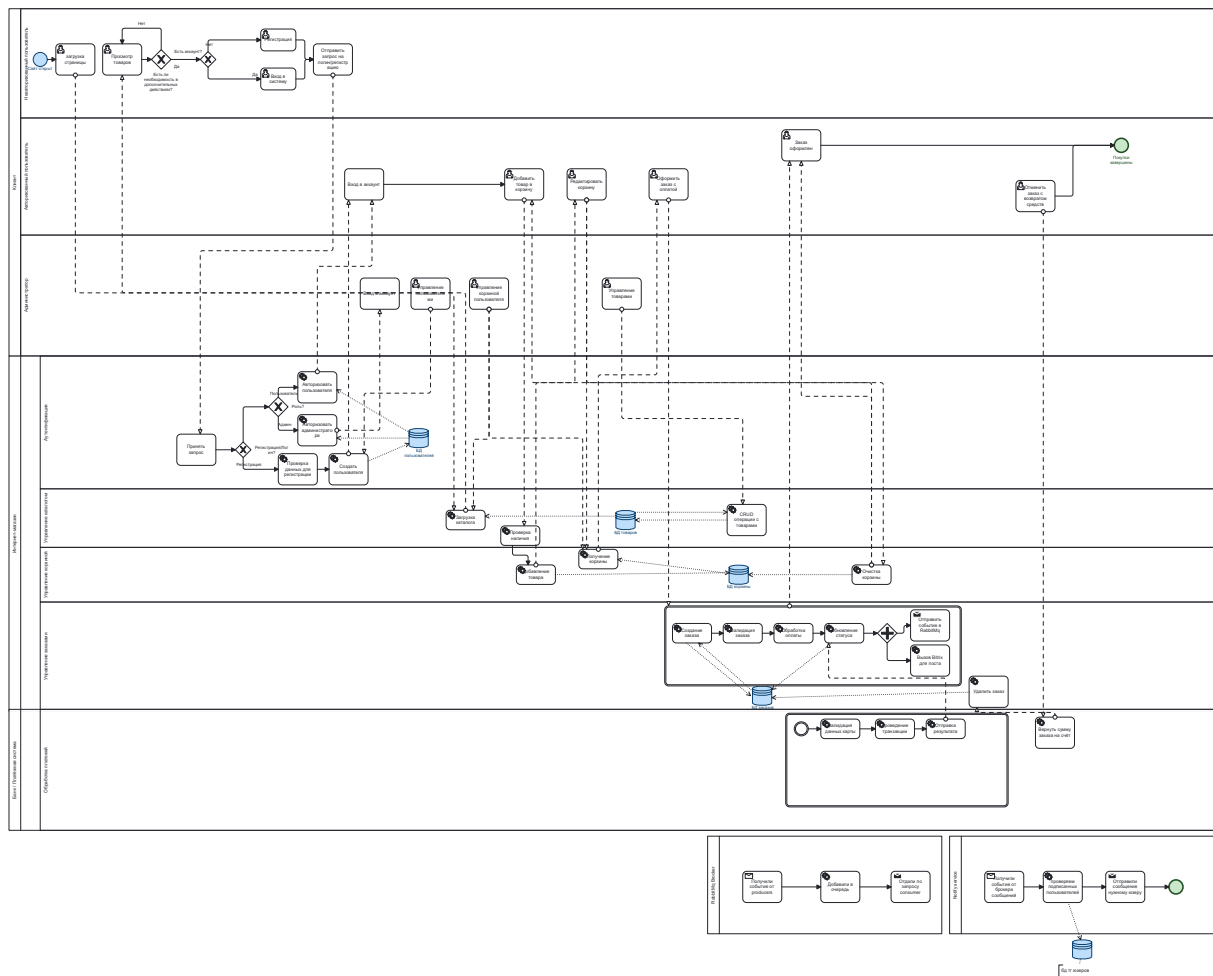


Рисунок 1: BPMN диаграмма

UML-диаграммы

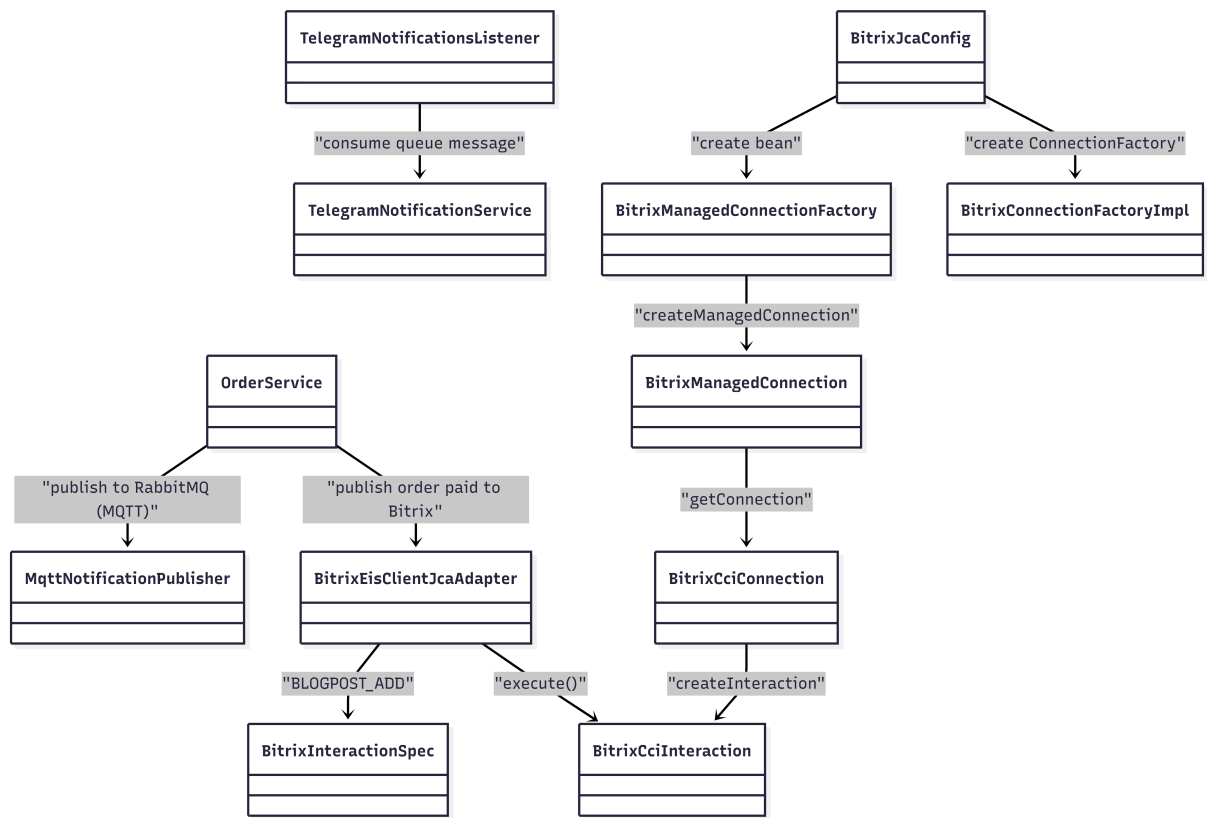


Рисунок 2: UML-диаграмма классов разработанных в этой лабе

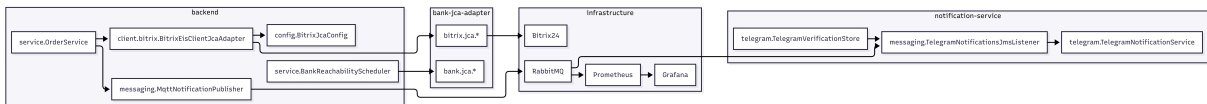


Рисунок 3: UML-диаграмма пакетов разработанных в этой лабе

Спецификация REST API

Спецификация REST API задана в формате OpenAPI 3.0.3 (файл api/openapi.yml).

Auth (без JWT)

Метод	Путь	Описание
POST	/auth/login	Вход. Тело: LoginRequest (login, password). Ответ: 200 LoginResponse (accessToken).
POST	/auth/register	Регистрация. Тело: RegisterRequest (login, email, password, name). Ответ: 201 MessageResponse.

Products (без JWT)

Метод	Путь	Описание
-------	------	----------

GET	/products	Каталог товаров. Ответ: 200 ProductListResponse: items, total, page, size.
GET	/products/{id}	Товар по идентификатору. Ответ: 200 Product (id, name, price).

Cart (Bearer JWT)

Метод	Путь	Описание
GET	/cart	Корзина текущего пользователя. Ответ: 200 CartResponse (items с itemId и product, total, page, size).
POST	/cart/items	Добавить товар в корзину. Тело: AddCartItemRequest (productId). Ответ: 201 MessageResponse.
DELETE	/cart/items/{itemId}	Удалить позицию корзины. Ответ: 200 MessageResponse.
DELETE	/cart/clear	Очистить корзину. Ответ: 200 MessageResponse.

Orders (Bearer JWT)

Метод	Путь	Описание
GET	/orders	Список заказов пользователя. Ответ: 200 OrderListResponse (items — OrderResponse, total, page, size).
POST	/orders/create	Создание заказа из корзины с оплатой в одной транзакции. Тело: PaymentRequest (cardNumber, expiresAt, cvv). Ответ: 201 OrderResponse; 400/500.
POST	/orders/{orderId}/cancel	Отмена заказа с возвратом средств. Ответ: 200 MessageResponse.

Admin Users (Bearer JWT; в спецификации ответ 403 при отсутствии прав администратора)

Метод	Путь	Описание
GET	/admin/users	Список пользователей. Query: page, size. Ответ: 200 UserListResponse; 401/403.
GET	/admin/users/{userId}	Пользователь по id. Ответ: 200 UserResponse; 401/403/404.
GET	/admin/users/{userId}/cart	Корзина пользователя. Ответ: 200 CartResponse; 401/403/404.
DELETE	/admin/users/{userId}/cart/clear	Очистить корзину пользователя. Ответ: 200 MessageResponse; 401/403/404.

Admin Orders (Bearer JWT; 403 при отсутствии прав администратора)

Метод	Путь	Описание
-------	------	----------

GET	/admin/orders	Все заказы. Query: page, size. Ответ: 200 OrderListResponse; 401/403.
POST	/admin/orders/{orderId}/cancel	Отмена заказа с возвратом. Ответ: 200 MessageResponse; 401/403/404.

Admin Products (Bearer JWT; 403 при отсутствии прав администратора)

Метод	Путь	Описание
GET	/admin/products	Список товаров (в т.ч. скрытые). Query: page, size. Ответ: 200 ProductListResponse; 401/403.
POST	/admin/products	Создать товар. Тело: CreateProductRequest (name, price). Ответ: 201 Product; 400/401/403.
PUT	/admin/products/{productId}	Обновить товар. Тело: UpdateProductRequest (name, price). Ответ: 200 Product; 400/401/403/404.
DELETE	/admin/products/{productId}	Удалить товар. Ответ: 200 MessageResponse; 401/403/404.

Выводы по работе

В ходе лабораторной работы приложение было доработано механизмами асинхронной и распределённой обработки данных с использованием RabbitMQ, MQTT и двух независимых consumer-узлов. Реализованы периодические задачи на базе Spring Scheduler для автоматизации служебных операций. Выполнена интеграция с внешней корпоративной системой Bitrix24 через технологию JCA. Также настроены мониторинг, контейнеризация и развёртывание системы на серверной инфраструктуре. В результате приложение стало более масштабируемым, отказоустойчивым и приближённым к реальным корпоративным требованиям.